Zero-Knowledge Proofs
University of Michigan, Winter 2026

Homework 2

Instructor: Paul Grubbs
Student: YOUR NAME HERE

This homework is due by **11:59pm on April 10** via the Autograder page. Start early!

Instructions. Solutions must be *typed* in LaTeX (a template for this homework is available on the course web page). Your work will be graded on *correctness*, *clarity*, and *concision*. You should only submit work that you believe to be correct; if you cannot solve a problem completely, you will get significantly more partial credit if you clearly identify the gap(s) in your solution. It is good practice to start any long solution with an informal (but accurate) "proof summary" that describes the main idea.

You may consult external sources. However, you must ***write your own solutions*** and ***list your sources*** for each problem. If you turn in a proof you cannot explain orally, you will fail the assignment.

1. *(The eq-extension lemma.)* Prove the following lemma, which we used in building the Delphian PIOP.

   Lemma. *Let* $g : \mathbb{F}^n \to \mathbb{F}$ *be a polynomial. Let* $\mathsf{eq} : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}$ *be the function that returns 1 if its two* n*-bit inputs are equal component-wise, and 0 otherwise, and let* $\widetilde{\mathsf{eq}}$ *be the multilinear extension of* eq*. If* $\exists\, y \in \{0,1\}^n$ *such that* $g(y) \neq 0$*, then*

$$\Pr_{r \leftarrow\$ \mathbb{F}^n}\left[\sum_{x \in \{0,1\}^n} \widetilde{\mathsf{eq}}(r,x)\, g(x) = 0\right] \leq \frac{n}{|\mathbb{F}|}\,.$$

   Note that the sum only evaluates g at Boolean inputs.

   Solution:

2. *(Verifiable pattern matching.)*

   In this problem you will construct a sumcheck-based protocol for verifiable pattern matching with wildcards. The protocol is based on a (non-cryptographic) randomized fingerprinting technique due to Adam Kalai, "Efficient pattern matching with don't-cares" (SODA 2002).

   Setup. Let $\mathbb{F}$ be a finite field. A *text* is a vector $T \in \mathbb{F}^N$ with $N = 2^n$. A *pattern* is a vector $P \in (\mathbb{F} \cup \{*\})^m$ with $m = 2^\ell$ for some $\ell \leq n$, where $*$ denotes a *wildcard* that matches any field element. We say P *matches* T at position j (for $0 \leq j \leq N - m$) if for every $i \in \{0, \ldots, m-1\}$, either $P_i = *$ or $T[j+i] = P_i$.

   The Kalai fingerprint. Given T and P, the following randomized test checks whether P matches T at any position. First, compute the vector $\vec{r} \in \mathbb{F}^m$ using the rule that $\vec{r}_i = 0$ if $P_i = *$ and $\vec{r}_i \leftarrow\$ \mathbb{F}$ otherwise. Compute the *pattern fingerprint* $t = \sum_{i \in \{0,\ldots,m-1\}:\, P_i \neq *} \vec{r}_i \cdot P_i$. Then, for each position j, compute the fingerprint $s(j)$ as

$$s(j) \;=\; \sum_{i=0}^{m-1} \vec{r}_i \cdot T[j+i]\,.$$

   Output match if $s(j) = t$ for any j and $\mathsf{mismatch}$ otherwise. (It is not needed for this problem, but a beautiful fact about the Kalai fingerprint is that $s(j)$ can be computed for all j in only $O(N \log m)$ field operations by expressing the vector of $s(j)$s as the convolution of the text and $\vec{r}$.)

(a) Prove that if P matches T at position j, the Kalai fingerprint always outputs match. Conversely, prove that if P does not match T at position j (i.e., there exists a non-wildcard position i_0 with $T[j+i_0] \neq P_{i_0}$), then

$$\Pr[s(j) = t] \;\leq\; \frac{1}{|\mathbb{F}|}\,.$$

Solution:

(b) A prover holds T and a commitment $C = \mathsf{Commit}(\widetilde{T})$ to the multilinear extension of T under a multilinear PCS. It also knows the opening γ of C. Both prover and verifier know C, P, and N. The prover wishes to convince the verifier that P matches T at a position j^* (which the prover discloses). Using the Kalai fingerprint and a single instance of the sumcheck protocol, construct a protocol that verifies P matches T at position j^*. The prover should send j^* to the verifier. Your protocol should require only *one* evaluation opening of the commitment C. State the degree per round, the number of rounds, the prover's work, and the verifier's work. Prove the soundness of your protocol.

Hint: define a "selector" vector $\alpha \in \mathbb{F}^N$, computable by the verifier from $\vec{r}$ and j^*, such that $\langle \alpha, T\rangle = s(j^*)$. Then express this inner product as a sum over the Boolean hypercube $\{0,1\}^n$ involving multilinear extensions.

Solution:

(c) At the conclusion of your protocol from part (b), the verifier will hold a random point $\rho \in \mathbb{F}^m$ and need the values $\widetilde{\alpha}(\rho)$ and $\widetilde{T}(\rho)$, where $\widetilde{\alpha}$ and $\widetilde{T}$ denote the multilinear extensions of α and T respectively. Show how the verifier can compute $\widetilde{\alpha}(\rho)$ itself without doing work proportional to the text size N. This makes our protocol succinct in both verifier computation and proof size (as long as the PCS is succinct).

Solution:

(d) *(Extra credit.)* Suppose the prover wishes to prove that a match exists *without revealing the position* j^*. Describe an extension of your protocol above that hides j^*. You need not prove rigorously that j^* is hidden, but you should sketch a convincing argument.

Solution:

3. *(Lariat: a lookup argument from log-derivatives.)*

A *lookup argument* lets a prover convince a verifier that every entry of a vector $\vec{a} \in \mathbb{F}^m$ appears in a *table* $\vec{t} \in \mathbb{F}^N$ (where all table entries t_j are distinct). In this problem you will analyze and then build Lariat, a lookup argument based on log-derivatives.

Setup and main idea. Let $\vec{t} = (t_1, \ldots, t_N) \in \mathbb{F}^N$ with all t_j distinct be a table, and let $\vec{a} = (a_1, \ldots, a_m) \in \mathbb{F}^m$ be a lookup vector. A *multiplicity vector* for $\vec{a}$ with respect to $\vec{t}$ is a vector $\vec{e} \in \mathbb{Z}_{\geq 0}^N$ where $e_j = |\{i \in [m] : a_i = t_j\}|$ counts how many times each table entry appears in $\vec{a}$.

The key algebraic observation behind Lariat is the following. If every a_i lies in $\vec{t}$ and $\vec{e}$ is the correct multiplicity vector, then the *log-derivative equation*

$$\sum_{i=1}^{m} \frac{1}{a_i - r} = \sum_{j=1}^{N} \frac{e_j}{t_j - r} \qquad (\star)$$

holds as an identity of rational functions in r. Conversely, as you will show below, if the lookup relation fails then (**??**) holds for at most $m + N$ values of $r \in \mathbb{F}$. So checking (**??**) at a random $r \leftarrow\!\!\$\, \mathbb{F}$ (with $r \notin \{a_1, \ldots, a_m\} \cup \{t_1, \ldots, t_N\}$) is a sound randomized test for the lookup relation. For parts (b) and (c), you may assume either that $\mathrm{char}(\mathbb{F}) > m$ or, equivalently, that multiplicities are interpreted as elements of $\mathbb{F}$ and "incorrect" means unequal in $\mathbb{F}$.

(a) Prove the following:
Lemma. *Let $\vec{a} \in \mathbb{F}^m$ and $\vec{t} \in \mathbb{F}^N$ with all t_j distinct. If there exists i^* such that $a_{i^*} \notin \{t_1, \ldots, t_N\}$, then for* any *choice of $\vec{e} \in \mathbb{F}^N$, equation (**??**) holds for at most $m + N$ values of $r \in \mathbb{F}$.*
*Hint: view both sides of (**??**) as rational functions in r and consider their poles.*

Solution:

(b) Now suppose that $a_i \in \{t_1, \ldots, t_N\}$ for all i, but the multiplicity vector is incorrect: $e_j \neq |\{i : a_i = t_j\}|$ for some j. Show that (**??**) still holds for at most $m + N$ values of $r \in \mathbb{F}$.
Hint: use partial fraction decomposition and compare residues at the poles $r = t_j$.

Solution:

(c) Now suppose $m = 2^\ell$ and $N = 2^n$. Design a PIOP (polynomial interactive oracle proof) that checks (**??**) using sumcheck. Work in the following model: the table $\vec{t}$ is public, the verifier has oracle access to the multilinear extensions of $\vec{a}$ and $\vec{e}$ (i.e., it can query $\widetilde{a}(\rho)$ and $\widetilde{e}(\rho)$ at any ρ of its choosing), and the prover may send additional oracle polynomials. Describe what auxiliary oracle polynomials the prover sends, what sumcheck claims are made, and how the verifier checks them. State the number of sumcheck rounds and the degree per round.

Solution:

(d) Give an alternative to the PIOP from part (c): design an R1CS instance that is satisfiable if and only if (**??**) holds for a given r. The R1CS witness should encode $\vec{a}$, $\vec{e}$, and any auxiliary values you need. Estimate the number of constraints needed.
Also explain how the prover can efficiently compute the full witness—including all auxiliary values—with a *single* multiplicative inverse computation in $\mathbb{F}$ (plus other cheaper operations). You may find it helpful to recall an optional task from Project 1.

Solution:

4. *A univariate PCS from a multilinear PCS.*

This question is due to Dan Boneh and Binyi Chen. Recall that a *multilinear polynomial commitment scheme* (multilinear PCS) lets one commit to a multilinear polynomial $g \in \mathbb{F}^{(\leq 1)}[X_1, \ldots, X_k]$—that is,

a polynomial of individual degree at most 1 in each of k variables—by computing a commitment C_g. Later, for any point $(r_1, \ldots, r_k) \in \mathbb{F}^k$, the committer can prove that $g(r_1, \ldots, r_k) = v$ for a claimed value $v \in \mathbb{F}$.

Suppose you are given such a multilinear PCS for polynomials in $\mathbb{F}^{(\leq 1)}[X_1, \ldots, X_k]$. Show how to use it to construct a *univariate* PCS for polynomials in $\mathbb{F}^{(\leq d)}[X]$ (univariate polynomials over $\mathbb{F}$ of degree at most d), where $d = 2^k - 1$.

(a) Explain how to commit to a polynomial $f \in \mathbb{F}^{(\leq d)}[X]$.
Hint: show how to map f to a multilinear polynomial $g \in \mathbb{F}^{(\leq 1)}[X_1, \ldots, X_k]$, and then commit to g using the multilinear PCS.

Solution:

(b) Explain how to open the committed polynomial at a point $x \in \mathbb{F}$. That is, given $x \in \mathbb{F}$ and a claimed value $y = f(x)$, describe a protocol between prover and verifier that proves $f(x) = y$ using only the opening procedure of the multilinear PCS.

Solution: